ModelicAI Studio: System Design

Nov 2, 2026 · @Anurag Dogra

Overview

ModelicAI Studio turns a plain-language request into a working, simulated Modelica model, and lets people inspect, change and re-run it in one browser workspace. It is built by Anurag Dogra (owner and maker) for students and researchers who model physical systems.

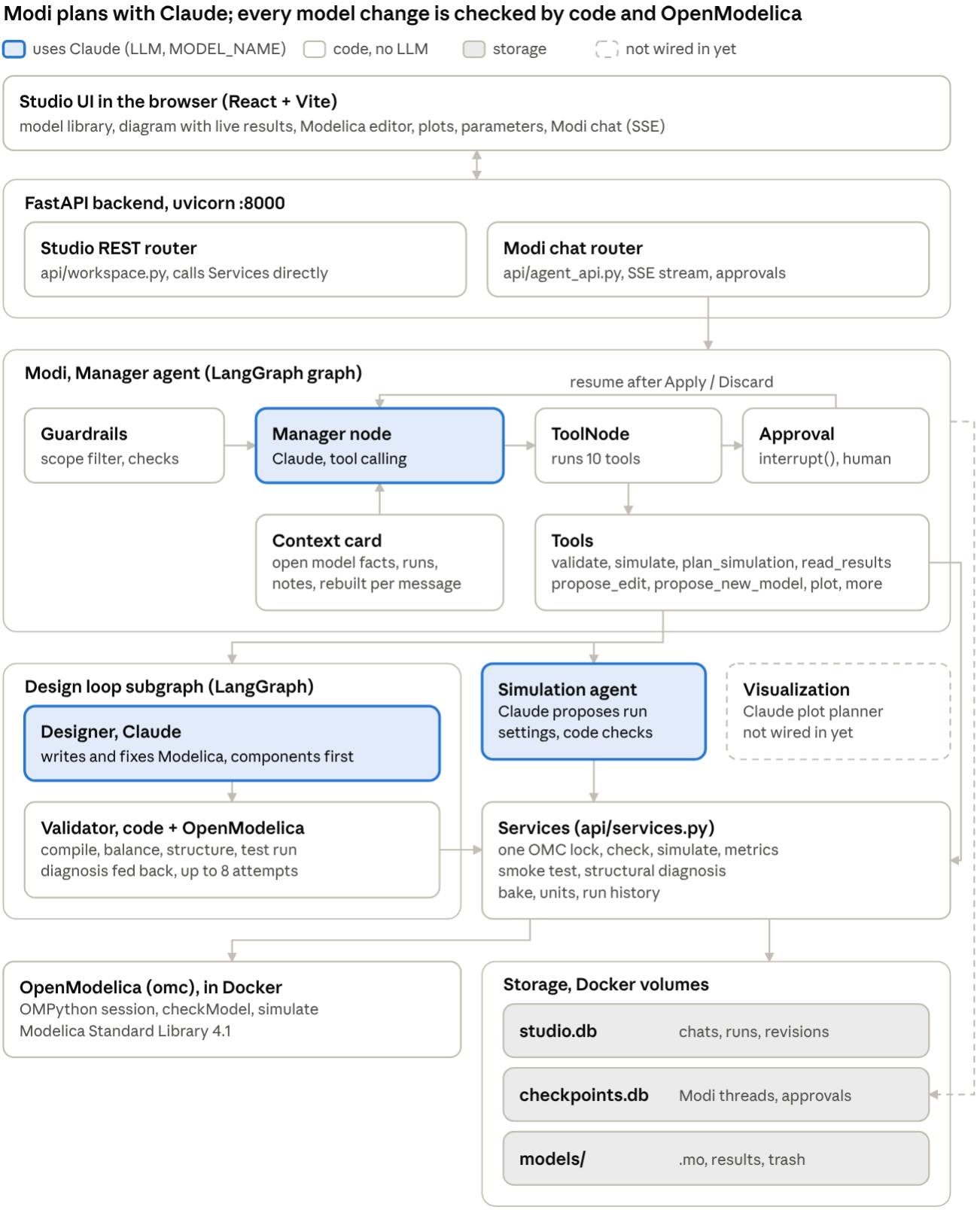
The product has two halves that share one OpenModelica engine:

- **The workbench:** model library, diagram with original Modelica Standard Library (MSL) icons, Modelica code editor, parameter panel, simulation setup, plots and live results on the diagram.
- **Modi, the modelling assistant:** a LangGraph agent that answers questions from the open model's facts, designs and edits models through a checked design loop, simulates, explains and fixes errors, and asks for approval before changing anything.

Design goals:

1. **Correct before shown:** a proposed model must compile, be balanced, be built from library components and pass a test run before the user sees it.
2. **Human in control:** structural changes and agent-chosen settings wait for Apply or Discard.
3. **Facts over guesses:** numbers come from result files, units from OpenModelica's model description, and error causes from a structural diagnosis.
4. **Memory per model:** every model keeps its chat, runs, revisions and notes across restarts.
5. **Focused scope:** Modi only helps with modelling and simulation in ModelicAI.

Architecture at a glance



agentic system design · 4 LLM agents, code checks, OpenModelica, 3 stores

Only four parts call an LLM, all Claude through `MODEL_NAME` (default `claude-haiku-5-5`): Modi's manager node, the designer, the simulation agent, and the visualization planner, which is not wired in yet. Everything else is code: the guardrails, the context card, the validator gates, the services and the OpenModelica calls. Modi's state is checkpointed to `checkpoints.db` (dashed line); the Studio REST router calls the services directly for GUI actions, with no LLM involved.

Frontend

The frontend is a Vite + React app in `frontend/`. `App.jsx` is the static layout shell; `studio.js` is an imperative engine started once from a React effect, which keeps the diagram, editor and plot fast without re-rendering React on every frame.

Area	What it does	Built with
Model library	List, open, `+ New` (fresh draft chat), delete (×)	REST
Diagram	OMC scene: MSL icons, placements, wires; pan, zoom, drag, select; dark canvas inverts icons	SVG
Live results	Value badges, flowing wires, DynamicSelect animation (e.g. tank level), time player, plot marker	SVG + Modelica expression evaluator
Modelica editor	Syntax highlighting, error-line marking from OMC messages, Check (⌘S)	CodeMirror 5
Results panel	Variable tree, plots with real units and two axes, pinned runs, CSV export	Plotly.js
Parameters	Run-time overrides, expressions (`2*k`) with live values, Write into model	REST + parser
Simulation setup	Start, stop, intervals, solver; numeric-only validation	—
Modi chat	Streaming steps, approval cards, Fix with Modi cards, per-model history	Server-Sent Events
Themes	11 themes + Auto (day/night), canvas toggle	CSS variables

During development `npm run dev` serves the UI on port 5173 and proxies `/api` to the backend; `npm run build` writes `static/`, which FastAPI serves at `/studio`.

Backend API

The backend is one FastAPI app (`main.py`, run by uvicorn on port 8000). At startup it builds the toolbox (`SimulationOrchestrator`: one OMC session, designer, simulation agent, graph parser), then mounts two routers and the static frontend. `/` redirects to `/studio/`; interactive API docs are at `/docs`.

Method + path	Purpose	Module
GET `/api/studio/models`	Model library with result status	workspace
GET / PUT / DELETE `/api/studio/models/{name}`	Read source + diagram fallback; save and check; delete (erase memory, move files to trash)	workspace
GET `/models/{name}/diagram`	Full scene from OMC: components, icons, wires, background, model icon	workspace
GET `/models/{name}/parameters`	Parameters and simulation options	workspace
POST `/models/{name}/run`	Simulate (no LLM), record the run, diagnosis on failure	workspace
GET `/models/{name}/variables`, `/results`, `/units`, `/runs`	Result variables, series, units, run history	workspace
POST `/models/{name}/bake`	Write parameter values or expressions into the source	workspace
GET `/models/{name}/messages`	Chat history of a model or draft	workspace
GET `/icon/{class}`	MSL icon proxy and cache (fallback view)	workspace
POST `/api/studio/agent`	Message to Modi, streamed back as SSE	agent_api
POST `/api/studio/agent/resume`	Apply, Discard or refine a proposal	agent_api
GET `/api/studio/agent/pending`	Open approval for a thread, if any	agent_api

All `/models/...` paths sit under `/api/studio`. Modi's stream emits typed events: `step` and `tool` (progress), `tool_result`, `approval` (a proposal card), `applied`, `ui` (plot, set parameters, refresh results), `failed` (opens a Fix with Modi card), `message`, `error` and `done`.

Modi, the Manager agent

Modi is a LangGraph `StateGraph` (`agents/manager.py`) with three nodes: **manager** (Claude with ten bound tools), **tools** (LangGraph `ToolNode` , errors returned as results instead of crashes) and **approval** (`interrupt()`). After tools run, the graph routes to approval when a tool returned a proposal, otherwise back to the manager; a reply without tool calls ends the turn.

On every message the manager receives: the system prompt and scope rules, the **context card** of the open model (components, parameters, last two runs with metrics, revisions, notes, origin, source) and the UI state (selected component, plotted variables, pending overrides), plus the last 30 messages of the thread.

Tool	What it does	Needs approval
`read_model`	Full source and card of the open or another model	No
`read_results`	Initial, final, min, max, time of max, 2% settling time per variable	No
`validate`	OMC check, plus structural diagnosis on failure	No
`simulate`	Run with given or last settings; diagnosis on failure	No
`plan_simulation`	Simulation agent chooses settings; checked, then proposed	Yes
`propose_edit`	Design loop on the current source; diff proposal	Yes
`propose_new_model`	Design loop from a description	Yes
`set_parameters`	Fills the parameter panel (run-time overrides)	No
`plot`	Selects variables in the plot	No
`remember_note`	Stores a lasting fact or goal for the model	No

Memory. A `SqliteSaver` checkpointer stores each thread in `data/checkpoints.db` , so a run paused for approval survives reloads and restarts. The UI chat history is kept separately in `studio.db` .

Design loop

The design loop (`agents/design_graph.py`) is the original designer → validator → retry `StateGraph` , run as a subgraph by `propose_edit` and `propose_new_model` . It works on a scratch copy in `models/.proposals/<id>/` , so the real model changes only after Apply.

Each attempt passes four gates in order; the first failure ends the attempt and its error goes back to the designer:

1. **Compiles:** OMC `checkModel` with full diagnostics from the error buffer.
2. **Balanced:** equation count equals variable count.
3. **Library components:** at least two MSL component instances, unless the user asked for equations or edits an existing equation model.
4. **Test run:** a short simulation up to the model's own stop time (200 points) catches initialization and runtime failures.

Failures on equation count, initialization or the test run carry the **structural diagnosis** (unconnected inputs, open fluid ports). If an attempt fails with the same error as the one before, the designer is told its last change did not help and must change approach. The loop allows `DESIGN_MAX_RETRIES` attempts (default 8, at most 10); after that Modi reports the failure instead of proposing a broken model.

Supporting agents and services

LLMs are used only where judgement is needed; everything that can be checked is checked by code.

Part	File	Uses an LLM	Role
Toolbox	`agents/orchestrator.py`	No	Creates the OMC session, designer, simulation agent and graph parser once
Designer	`agents/designer.py`	Yes	Writes and fixes Modelica, component-first, with an MSL cheat sheet and layout rules
Simulation agent	`agents/simulation.py`	Yes	Proposes stop time, intervals, solver, overrides from the model's physics
Visualization planner	`agents/visualization.py`	Yes	Plans plot layouts (kept; not wired in yet)

Part	File	Uses an LLM	Role
Context card	`agents/context.py`	No	Builds the open model's facts for Modi on every message
Guardrails	`agents/guardrails.py`	No	Scope filter and output check
Services	`api/services.py`	No	OMC operations under one lock: check, simulate, metrics, smoke test, diagnosis, apply code
Bake	`api/bake.py`	No	Paren-aware rewrite of parameter values and expressions in source

Simulation agent checks. Its JSON answer must have finite numbers, stop greater than start, 10 to 100,000 intervals, a known solver, and overrides only for existing parameters with numeric values. An invalid answer is sent back once with the reasons; a second failure falls back to the model's own `experiment` annotation. The result then passes the same `validate_settings` gate as every run.

Structural diagnosis. `Services.diagnose()` reads components and connections through OMC and lists unconnected connectors, ranked: causal inputs first (the usual cause of "too few equations"), then open fluid ports, then other physical connectors. It feeds the design loop, Modi's validate/simulate/propose_edit tools and the Simulate error card.

OpenModelica integration and diagram engine

One OMPython `OMCSessionLocal` runs inside the container as the non-root user `modelica` (OMC refuses to start as root). Every call that touches it holds `omc_lock`, so the web API, Modi and the design loop never interleave commands.

`utils/omc_client.py` loads the Modelica Standard Library at startup and runs `compile_model`: `loadFile` + `checkModel`, reading OMC's error buffer for real diagnostics. A model reported as successful but with different equation and variable counts is treated as failed.

`utils/omc_graphics.py` builds the diagram from OMC's graphical API, the same calls OMEdit uses:

- `getComponents` and `getComponentAnnotations` give instances and their Placement; `getNthConnection` and `getNthConnectionAnnotation` give wires and routes.

- `getIconAnnotation` (with inherited classes) gives each icon's primitives, rendered to SVG in Modelica coordinates; connector icons are placed at their icon transformation, which also yields port positions and types.
- `DynamicSelect(static, dynamic)` keeps both parts: the static one is drawn, the dynamic expression becomes a `data-dyn-*` attribute that the browser evaluates during playback (fills, text, colours, visibility, points).
- Text placeholders (`%name` , `%R`) are filled in the browser from component modifiers, resolved through model parameters (`c=k` shows `c=10`).
- Wires without routes get orthogonal routes that leave each pin in its facing direction.

Results and units. Results come from OMC's v4 `.mat` file (`read_series` , `read_variable_names`); units come from `<Name>_init.xml` . Base units are used, because result files store base units (K, not degC).

Data and persistence

All state lives in two folders, both mounted as Docker volumes and both git-ignored. Nothing is kept in process memory that matters after a restart.

Store	Location	Contents	Keyed by
Model files	`models/<Name>/`	`.mo` source, OMC build output, `_res.mat` , `_init.xml` , `graph_schema.json`	model name
Proposals	`models/.proposals/<id>/`	Scratch copies compiled and test-run by the design loop	proposal id
Trash	`models/.trash/<Name>-<time>/`	Files of deleted models (no memory attached)	name + time
`messages`	`data/studio.db`	Chat history shown in the UI, proposals, decisions	model or `_new_<id>` draft
`runs`	`data/studio.db`	Settings, overrides, status, key metrics, wall time, source (GUI or agent)	model
`revisions`	`data/studio.db`	Every saved version of the code with a one-line summary	model

Store	Location	Contents	Keyed by
`notes`	`data/studio.db`	Facts and goals the user told Modi to remember	model
`proposals`	`data/studio.db`	Proposed edits, new models and run settings, with status	proposal id
`model_meta`	`data/studio.db`	Origin prompt and purpose of each model	model
Checkpoints	`data/checkpoints.db`	Modi's LangGraph state, including runs paused for approval	thread id

Thread ids. An open model's thread is its name; each "+ New" opens a fresh `_new_<random>` draft. A model created from a draft gets its own thread, seeded with the original prompt.

Deletion. Deleting a model erases its rows in every table above and its checkpoint thread, removes its proposal folders, and moves its files to the trash. A new model with the same name starts with no memory.

Key flows

New model from a sentence

1. The user clicks "+ New" and writes a request; the UI posts it to `/api/studio/agent` with a fresh draft thread.
2. The input filter passes it; Modi calls `propose_new_model`.
3. The design loop writes, checks, balance-checks, structure-checks and test-runs the model in a scratch folder, streaming each step.
4. The run pauses on an approval card. On Apply the model is saved, a revision recorded, and the UI opens the new model with its own chat.

Edit with approval

1. The user asks for a change ("add a second spring"). Modi calls `propose_edit` with the current source.
2. The design loop produces a checked version; the card shows the diff.
3. Apply saves it; Discard changes nothing; a chat reply refines the proposal and loops back.

Simulate

1. The GUI Simulate button checks the inputs, saves unsaved code via Check, then posts settings and overrides to `/run`. No LLM is involved.
2. The backend simulates, computes metrics, records the run and returns the variables.
3. The UI loads units, refreshes plots and starts the live diagram view on the final values.

Fix an error

1. A failed Check or Simulate (GUI or Modi) shows a red card with the cleaned OMC message and the structural diagnosis.
2. "Fix with Modi" sends the error; Modi re-validates, attaches the diagnosis as facts, and calls `propose_edit`.
3. The design loop retries up to 8 times, escalating repeated errors, and proposes a fix only once it compiles, balances and runs. Modi then re-validates and re-simulates.

Guardrails and safety

Modi only helps people use ModelicAI; nothing changes a model without a person approving it.

Layer	Where	What it stops
Input filter	`guardrails.check_input`, before the LLM	Competitors and their vendors, other companies and AI products, outside topics, general code, translation, jailbreak phrases; refused with no tokens spent
Scope rules	Modi's system prompt	Comparisons, naming other products, revealing instructions; says it is Modi, built by Anurag Dogra
Output check	`guard_reply` after the LLM	Replies naming competitors or companies, non-Modelica code blocks
No code tools	Tool list	Modi cannot run code or write files except through proposals
Approvals	`interrupt()`	Structural edits, new models and agent-chosen settings wait for Apply

Layer	Where	What it stops
Input validation	UI and server	Non-numeric setup values, stop ⩽ start, bad interval counts, unknown solvers, unsafe parameter expressions
Write limits	`api/bake.py`, `services`	Only numbers, booleans or clean expressions written into source; overrides sanitised before reaching OMC

Allowed by design: Modelica, the MSL, OpenModelica (the product's own engine), SI units such as "tesla", and the physics needed to model a system. The pasted OMC error inside a "Fix this … error" message is treated as data, not as a request.

Deployment and configuration

The product ships as one Docker image (Ubuntu 22.04, OpenModelica from the official apt repository, MSL 4.1 installed for the `modelica` user, Python 3.10). The frontend is built beforehand and copied in as `static/`.

```
cd frontend && npm install && npm run build && cd ..
docker build -t modelicai .
docker run --rm -p 8000:8000 -e ANTHROPIC_API_KEY=... \
  -v "$PWD/data":/app/data -v "$PWD/models":/app/models modelicai
```

Variable	Default	Purpose
`ANTHROPIC_API_KEY`	none (required)	Claude access for Modi, designer, simulation agent
`MODEL_NAME`	`claude-haiku-5-5`	Claude model for all agents (no `temperature` passed)
`DESIGN_MAX_RETRIES`	8 (max 10)	Attempts per design request
`STUDIO_DB`	`data/studio.db`	Chats, runs, revisions, notes, proposals
`STUDIO_CHECKPOINT_DB`	`data/checkpoints.db`	Modi's LangGraph checkpoints

Development. Run the backend in the container on port 8000 and `npm run dev` in `frontend/` on port 5173; Vite proxies `/api` to the backend. `models/` and `data/` are git-ignored; secrets live in `.env*` files, also ignored.

Limitations and roadmap

Known limitations

- One OMC session behind one lock: simulations and agent runs queue, one at a time.
- Single user, no accounts; anyone who reaches port 8000 sees every model.
- The diagram is view-only: components cannot be dragged in from a library or wired by hand yet.
- Most automated testing so far used stand-ins for OpenModelica and Claude; there is no test suite against the real engine.
- The Dockerfile does not build the frontend; `static/` must be built first.
- The visualization planner exists but is not wired in.

Roadmap

1. Commit the current work on a branch; add the frontend build and a `.dockerignore` to the Dockerfile.
2. A test suite run inside the container against real OpenModelica: open, check, simulate, diagram, bake, design loop with a broken model, guardrail prompts.
3. Mascot (Spring-Owl or elephant) as logo, Modi's avatar and state animations; a first-run welcome.
4. Drag-and-drop editing: MSL library browser with original icons, place components, draw wires, code updated automatically.
5. Parameter sweeps and run comparisons set up by Modi from one sentence; an explain mode for students; FMU and report export.
6. Accounts, per-user workspaces, sharing by link, and several OMC sessions in parallel.